

Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1[†]

*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan

[†]Affiliation 2

Email: crg@mail.ntust.edu.tw

Abstract—nFAPI-based Split Option 6 separates the MAC scheduler and High-PHY across packet-switched transport, where scheduling messages must arrive at the PNF within a strict timing window. Existing static slots-ahead scheduling either over-provisions lead time, increasing MAC HARQ round-trip time (RTT), or under-provisions it, causing late arrivals, missing-slot events, and throughput degradation under network jitter. This paper proposes an adaptive timing control framework for nFAPI Split 6 that dynamically adjusts the scheduler trigger offset using PNF Timing Info feedback. The proposed controller applies EWMA-based arrival-margin and jitter estimation to increase slots ahead rapidly under timing risk and decrease it conservatively under stable conditions. We implement the framework in an OpenAirInterface-based end-to-end testbed with a commercial O-RU and COTS UE. Experimental results show that the proposed method maintains stable throughput under injected delay and jitter while avoiding unnecessary MAC HARQ RTT inflation compared with static slots-ahead baselines.

Index Terms—nFAPI, O-RAN, Functional Split, Adaptive Control.

I. INTRODUCTION

A. Background and Motivation

In nFAPI-based Split Option 6, the MAC scheduler in the VNF must send DL_TTI.request messages sufficiently ahead of the target slot so that the PNF can process them within its timing window. If the message arrives later than the deadline, the PNF discards the scheduling request, resulting in missing-slot events and throughput degradation.

Existing OAI nFAPI-style implementations rely on statically configured slots-ahead scheduling. Such a static configuration creates an inherent throughput-latency trade-off: a larger slots-ahead value improves robustness against delay and jitter but increases MAC HARQ RTT, whereas a smaller value reduces latency but becomes vulnerable to late packet arrivals.

However, existing open-source implementations such as OAI have historically relied on simplified slot-indication forwarding and static slots-ahead configuration, leaving the SCF-defined Timing Info based delay-management loop incomplete.

To resolve this trade-off, this paper proposes an adaptive timing control framework for nFAPI Split 6. Our approach dynamically adjusts the scheduler trigger offset based on closed-loop feedback using PNF Timing Info reported by the PNF. By applying an Exponentially Weighted Moving Average (EWMA) model, the controller estimates the real-time arrival

margin and transport jitter. The controller then applies a fast-increase and conservative-decrease policy to dynamically update the slots ahead, securing system stability under congestion while optimizing latency under stable network conditions.

This paper addresses the following problem: how to maintain nFAPI Split 6 scheduling stability over non-deterministic packet-switched transport without relying on hardware-level synchronization or static worst-case over-provisioning. The key challenge is that the scheduler must choose a slots-ahead value before observing the future network delay. A conservative value improves robustness but increases MAC HARQ RTT, while an aggressive value reduces latency but risks late packet arrivals and slot loss. To resolve this trade-off, we propose a closed-loop delay management mechanism that uses PNF Timing Info to estimate the arrival margin and adapt the slots-ahead value online.

B. Related Works

Recent advancements in Open RAN (O-RAN) architectures have driven significant research into optimizing split-architecture performance and managing network latency. These works can be broadly classified into three categories.

Hardware-assisted Split 7.2: High-performance testbeds such as XSG [1] offload heavy physical layer processing to dedicated GPUs and Mellanox SmartNICs. While achieving commercial-grade peak throughput, these implementations depend on specialized, expensive hardware acceleration and strict synchronization infrastructure (e.g., hardware-level PTP over TSN). This severely limits their deployment flexibility in general-purpose, non-deterministic packet-switched networks.

Intra-node Software Scheduling: At the intra-node computational level, Lee *et al.* [2] proposed an adaptive Layer 1 transmission strategy inside FAPI-based gNBs, while Aquifer [3] optimized OS-level scheduling microservices to absorb processing delay spikes. However, these methods only address intra-node delays and do not handle cross-node timing violations caused by unpredictable fronthaul network transport delay and jitter.

nFAPI Delay Management Gap: Although the Small Cell Forum (SCF) nFAPI specification [4] defines standard interface messages for Node Sync and Delay Management, existing open-source Split 6 platforms, including OpenAirInterface (OAI), only implement static slots-ahead trigger configurations. This leaves the spec-defined closed-loop dynamic timing

adjustment incomplete. Our proposed EWMA-based adaptive trigger offset controller fills this critical gap, providing a cost-effective, software-only solution for COTS servers.

Specifically, compared to Split 7.2 implementations like X5G [1] that require hardware-level PTP synchronization and GPU/SmartNIC offloads, or FAPI-based Split 6 designs [2] restricted to intra-node shared memory and process-level adjustments, our proposed framework operates at the slot/TTI level on standard COTS servers, actively managing cross-node network jitter and delay over standard packet-switched Ethernet without hardware acceleration dependencies.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

To address the limitations of static configurations under such non-deterministic transport environments, this paper introduces a dynamic timing adjustment framework that achieves closed-loop synchronization stability. The main contributions of this paper are summarized as follows:

- **SCF-compliant nFAPI timing framework for OAI Split 6:** We redesign the OAI nFAPI timing path to support Node Sync, PNF Timing Info, and Delay Management, enabling closed-loop timing control instead of simplified static slot forwarding.
- **EWMA-based adaptive slots-ahead control:** We propose a lightweight feedback controller that estimates the arrival margin and jitter from PNF Timing Info and dynamically adjusts the scheduler trigger offset to balance throughput robustness and MAC HARQ RTT.
- **End-to-end validation under non-deterministic transport:** We implement the proposed method on an OpenAirInterface-based E2E physical testbed with a commercial Pegatron O-RU and COTS UE. We evaluate throughput, MAC HARQ RTT, missing-slot behavior, and stability under injected delay and jitter. The implementation has been successfully merged into the official upstream OAI codebase to support reproducibility and future research.

II. SYSTEM MODEL

Before diving into the system details, this study introduces the adopted system framework. To integrate the Network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the proposed system adopts a stable two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

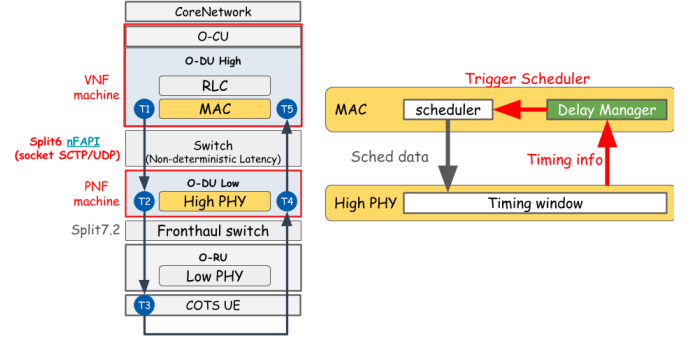


Fig. 1. System architecture and delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- **Core Network (CN):** The central element responsible for routing, session management, and authenticating user data.
- **O-RAN Central Unit (O-CU):** A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- **O-RAN Distributed Unit (O-DU):** To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- **Switch:** A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- **O-RAN Radio Unit (O-RU):** A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- **Commercial Off-The-Shelf User Equipment (COTS UE):** Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC, Scheduler, and Timing Core

Within the O-DU High virtualized network function, the timing control is organized as a unified timing control system. The core processes handling this timing core include the Scheduler, Node sync, and Delay Management:

- **Scheduler:** Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.

- **Node sync:** Responsible for software-level clock synchronization. This procedure periodically exchanges synchronization timestamps ($t_1 \sim t_4$) with the PNF via standard *DL/UL_node_sync* messages. It calculates the instantaneous clock offset and applies slot-level (S_{adj}) and microsecond-level (μs_{adj}) adjustments to align the VNF's local clock to the PNF baseline, eliminating long-term hardware clock drift.
- **Delay Management:** Operates on top of the stabilized clock phase established by Node sync. This mechanism utilizes the PNF Timing Info (Δt_{arrive}) feedback from the PNF to dynamically estimate transport network jitter. It then actively adjusts the slots ahead (S_{ahead}) trigger offset of the scheduler. This control compensates for the non-deterministic packet transport delays and VNF processing delays without inducing synchronization loss.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined, strict time duration. Let T_{window} denote the duration of this PNF timing window, and T_{slot} denote the subframe slot duration. When the scheduling message (specifically the *DL_TTI.request*) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the slot data.
- **Timing Info (Δt_{arrive}):** The PNF continuously monitors the exact timing info of control messages relative to the transmission deadline. This information, denoted as Δt_{arrive} , represents the remaining time margin within the timing window. The PNF periodically sends this information back to the MAC layer for Delay Management. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Timing Model

Let T_{slot} denote the slot duration and S_{ahead} denote the number of slots by which the VNF scheduler triggers earlier than the target slot. For a scheduling message generated at VNF time t_g , the corresponding target PNF processing deadline is determined by the target slot boundary. The arrival margin reported by the PNF is denoted by Δt_{arrive} , where $\Delta t_{arrive} < 0$ indicates that the message arrives before the deadline, and $\Delta t_{arrive} > 0$ indicates a late arrival. Therefore, the delay-management objective is to select the minimum stable S_{ahead} such that late arrivals are avoided ($\Delta t_{arrive} \leq 0$) while MAC HARQ RTT is not unnecessarily inflated.

E. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal

environments, this study defines the following Key Validation Metrics:

- **MAC Layer HARQ RTT ($T_5 - T_1$):** Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. This metric accurately reflects the feedback efficiency and operational health of the link layer.
- **VNF-PNF DL Latency ($T_2 - T_1$):** Measures the one-way downlink latency accumulated from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

Note that RTT denotes the Round-Trip Time. To validate the split-architecture stability under non-ideal fronthaul conditions, network latency simulation is conducted at the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments via Linux Traffic Control (TC).

To validate the proposed performance optimization within a realistic network environment, the system design leverages a commercial-grade deployment framework:

- **O-RAN Split 7.2x Interoperability:** While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [5], commercial-grade Open Radio Units (O-RUs) natively support the O-RAN Split Option 7.2x. To achieve seamless integration with open-source stacks like OpenAirInterface [6], our system bridges these interfaces.
- **Hybrid System Architecture for E2E Validation:** To validate the nFAPI performance over a fully commercial ecosystem, this paper adopts a hybrid architecture (Split Option 6 + Split Option 7.2). This setup establishes a translation/interworking layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with a commercial Pegatron O-RU, facilitating end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. PROPOSED METHOD

This section details the proposed dynamic timing adjustment framework. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose two core timing control algorithms: Node sync for software-level clock synchronization and Delay Management for slot-level adaptive delay management. The timing control of this system is organized as a closed-loop system, ensuring that the VNF-side scheduling can dynamically track and compensate for both clock drift and slot-level transport latency variation. The two algorithms cooperate as follows:

- 1) **Node sync:** Maintains phase and slot boundary alignment between the VNF and PNF. It periodically exchanges timestamps to calculate the absolute phase offset, correcting for hardware clock drift.
- 2) **Delay Management:** Tracks and estimates packet transport delays and jitter at the slot level, dynamically adjusting the slots ahead (S_{ahead}) to guarantee that control messages fall within the PNF's strict timing window.

A. Node sync

To perform software-level clock synchronization, the system adopts a timestamp exchange mechanism. The VNF periodically transmits a downlink node synchronization message with a local timestamp t_1 . Upon receiving the message, the PNF records the local arrival timestamp t_2 and transmits an uplink node synchronization message back to the VNF, including the transmit timestamp t_3 and the echoed t_2 . When the VNF receives the message, it records its local arrival timestamp t_4 . Because the nFAPI timestamps are constrained by a 1024 subframe number (SFN) loop, they wrap around every 10.24 seconds (10240000 μ s). The algorithm calculates the differences $d_1 = t_2 - t_1$ and $d_2 = t_4 - t_3$, applying a wrap-around correction to align them within the interval $[-5120000, 5120000]$ μ s. The VNF then calculates the instantaneous clock offset:

$$\text{offset} = \frac{d_1 - d_2}{2} \quad (1)$$

To prevent timing adjustments from causing control loop oscillations, the synchronization control loop implements a state-based locking mechanism. The system has two synchronization states: locked ($\text{sync_locked} = 1$) and unlocked ($\text{sync_locked} = 0$). In the locked state, the VNF monitors the drift of the local clock. If the absolute offset exceeds the slot duration T_{slot} :

$$|\text{offset}| > T_{\text{slot}} \quad (2)$$

the VNF unlocks the synchronization ($\text{sync_locked} = 0$) to trigger re-synchronization. In the unlocked state, the VNF checks whether the offset has converged within the slot duration T_{slot} . If $|\text{offset}| \leq T_{\text{slot}}$, the system locks the synchronization state ($\text{sync_locked} = 1$). Otherwise, the VNF decomposes the offset into a slot-level adjustment S_{adj} and a microsecond-level adjustment μs_{adj} based on the slot duration T_{slot} :

$$S_{\text{adj}} = \text{offset} / T_{\text{slot}} \quad (3)$$

$$\mu s_{\text{adj}} = \text{offset} \pmod{T_{\text{slot}}} \quad (4)$$

These adjustments are accumulated into the global adjustment variables:

$$\text{slot_adjustment} \leftarrow \text{slot_adjustment} + S_{\text{adj}} \quad (5)$$

$$\text{us_adjustment} \leftarrow \text{us_adjustment} - \mu s_{\text{adj}} \quad (6)$$

The negative sign in the microsecond adjustment reduces the local thread sleep time when the VNF clock is behind, speeding up the slot transition rate to catch up. The synchronization process is summarized in Algorithm 1.

B. Delay Management

To counteract changing network latency and maintain scheduling stability, a dynamic delay management strategy is implemented. This strategy operates as a closed-loop feedback controller based on standard Small Cell Forum (SCF) PNF Timing Info reported by the PNF. By evaluating the arrival margin relative to the PNF deadline, the controller dynamically adjusts the scheduling lead time S_{ahead} .

Algorithm 1 Node sync

Input: t_1, t_2, t_3 (Timestamps from PNF *UL_node_sync*), t_4 (VNF local receive timestamp)

Output: *slot_adjustment* (Slot adjustment accumulator), *us_adjustment* (Microsecond adjustment accumulator)

Notation:

offset (Instantaneous timing offset), *sync_locked* (Synchronization lock state)

T_{slot} (Slot duration in μ s), T_{wrap} (Wrap-around period, 10240000 μ s)

WrapCorrection(x, T) (Corrects x within $[-T/2, T/2]$)

Phase I: Timing Offset Calculation

1: $d_1 \leftarrow \text{WrapCorrection}(t_2 - t_1, T_{\text{wrap}})$

2: $d_2 \leftarrow \text{WrapCorrection}(t_4 - t_3, T_{\text{wrap}})$

3: $\text{offset} \leftarrow (d_1 - d_2) / 2$

Phase II: Hysteresis Lock and Clock Adjustment

4: **if** $\text{sync_locked} = 1 \wedge |\text{offset}| > T_{\text{slot}}$ **then**

5: $\text{sync_locked} \leftarrow 0$ ▷ Unlock due to excessive clock drift

6: **end if**

7: **if** $\text{sync_locked} = 0$ **then**

8: **if** $|\text{offset}| \leq T_{\text{slot}}$ **then**

9: $\text{sync_locked} \leftarrow 1$ ▷ Lock synchronization

10: **else**

11: $S_{\text{adj}} \leftarrow \text{offset} / T_{\text{slot}}$ ▷ Integer division

12: $\mu s_{\text{adj}} \leftarrow \text{offset} \pmod{T_{\text{slot}}}$

13: $\text{slot_adjustment} \leftarrow \text{slot_adjustment} + S_{\text{adj}}$

14: $\text{us_adjustment} \leftarrow \text{us_adjustment} - \mu s_{\text{adj}}$

15: **end if**

16: **end if**

17: **return** *slot_adjustment*, *us_adjustment*

1) *EWMA-based Arrival-Margin Estimation:* To smooth high-frequency network noise while remaining sensitive to timing drift, the controller maintains a running mean of the PNF Timing Info (μ_{arrive}) and the estimated jitter ($\text{dev}_{\text{arrive}}$). Let $\Delta t_{\text{arrive},i}$ be the observed signed PNF Timing Info in scheduling slot i . The EWMA updates for mean arrival margin and jitter are defined as:

$$\mu_{\text{arrive},i} = (1 - \alpha)\mu_{\text{arrive},i-1} + \alpha\Delta t_{\text{arrive},i} \quad (7)$$

$$\text{dev}_{\text{arrive},i} = (1 - \beta)\text{dev}_{\text{arrive},i-1} + \beta|\Delta t_{\text{arrive},i} - \mu_{\text{arrive},i}| \quad (8)$$

where the smoothing factors are chosen as $\alpha = 1/8$ to track the long-term trend, and $\beta = 1/4$ to capture fast jitter variations. These parameters are designed as negative powers of two ($\alpha = 2^{-3}$, $\beta = 2^{-2}$) to allow the controller to perform fast bitwise shift operations in real time.

The combined timing risk indicator R_i is evaluated to determine whether the arrival margin crosses the deadline boundary:

$$R_i = \mu_{\text{arrive},i} + \text{dev}_{\text{arrive},i} \quad (9)$$

A positive timing risk ($R_i > 0$) implies that the estimated worst-case arrival margin crosses the deadline, indicating an

anomaly condition, whereas a negative risk indicates a safe margin.

2) *Adaptive Slots-Ahead Control*: The update of the slots ahead trigger timing S_{ahead} is governed by a fast-increase and conservative-decrease control law to optimize latency while preserving throughput under dynamic congestion:

- **Rapid Increase (Preserve Throughput)**: If an anomaly timing risk is detected ($R_i > 0$), indicating that the scheduling message is crossing the PNF processing deadline, the controller immediately increases the slots ahead value to prevent late packet arrivals and missing-slot dropouts:

$$S_{\text{ahead},i+1} = \min \left(S_{\text{ahead},i} + \left\lceil \frac{R_i}{T_{\text{slot}}} \right\rceil, S_{\text{max}} \right) \quad (10)$$

and the stable observation counter C_{stable} is reset to 0. Here, $S_{\text{max}} = \lfloor T_{\text{window}}/T_{\text{slot}} \rfloor$ represents the maximum allowed slots ahead.

- **Gradual Decrease (Optimize Latency)**: If the system is under stable and safe conditions ($R_i < -dev_{\text{arrive},i}$), indicating that there is a sufficient timing margin, the controller decrements the slots ahead value by one slot once the stability condition is satisfied ($C_{\text{stable}} \geq C_{\text{threshold}}$):

$$S_{\text{ahead},i+1} = \max(S_{\text{ahead},i} - 1, S_{\text{min}}) \quad (11)$$

and resets $C_{\text{stable}} = 0$ to prevent premature timing reductions. Here, $S_{\text{min}} = 1$ is the minimum slots ahead trigger offset.

- **Maintain Baseline**: Otherwise, the trigger timing remains unchanged:

$$S_{\text{ahead},i+1} = S_{\text{ahead},i} \quad (12)$$

and the stable counter C_{stable} is incremented if the current sample is safe ($R_i \leq 0$).

The core delay management strategy is condensed in Algorithm 2.

C. Coordination between Node sync and Delay Management

To ensure stable operation under real-world conditions, Node sync and Delay Management must coordinate without inducing control loop oscillations. The microsecond-level Node sync focuses strictly on correcting clock phase offsets to align the VNF's scheduling thread with the PNF's slot transitions. Because it directly shifts the VNF's time base, any clock adjustment ($S_{\text{adj}}, \mu S_{\text{adj}}$) temporarily shifts the arrival timestamp reference. Delay Management is designed to absorb this transient shifting. By relying on history-weighted EWMA-based risk tracking (R_i), Delay Management avoids reacting to the temporary timing offsets caused by clock adjustments, allowing Node sync to converge. Once Node sync locks synchronization ($\text{sync_locked} = 1$), the time base stabilizes, allowing Delay Management to achieve precise slot-level optimizations (S_{ahead}).

Algorithm 2 Delay Management

Input: Δt_{arrive} (Observed PNF Timing Info), S_{curr} (Current trigger timing), T_{window} (Timing window), T_{slot} (Slot duration)

Output: S_{next} (Trigger timing for the next MAC schedule)

Notation:

μ_{arrive} (smoothed mean PNF Timing Info), dev_{arrive} (smoothed arrival jitter)

R (Timing risk indicator), C_{stable} (Stable observation counter)

$C_{\text{threshold}}$ (Stability observation threshold), $S_{\text{max}}, S_{\text{min}}$ (Boundary limits)

α, β (Smoothing factors for latency and jitter)

Phase I: Latency and Jitter Estimation

1: $\mu_{\text{arrive}} \leftarrow (1 - \alpha)\mu_{\text{arrive}} + \alpha\Delta t_{\text{arrive}}$ $\triangleright$ Update mean PNF Timing Info

2: $dev_{\text{arrive}} \leftarrow (1 - \beta)dev_{\text{arrive}} + \beta|\Delta t_{\text{arrive}} - \mu_{\text{arrive}}|$ $\triangleright$ Update arrival jitter

3: $R \leftarrow \mu_{\text{arrive}} + dev_{\text{arrive}}$ $\triangleright$ Evaluate combined risk indicator

Phase II: Fast-Up and Slow-Down Timing Adjustment

4: **if** $R > 0$ **then**

5: $S_{\text{next}} \leftarrow \min(S_{\text{curr}} + \lceil R/T_{\text{slot}} \rceil, S_{\text{max}})$ $\triangleright$ Rapid Increase

6: $C_{\text{stable}} \leftarrow 0$

7: **else if** $R < -dev_{\text{arrive}}$ **then**

8: **if** $C_{\text{stable}} \geq C_{\text{threshold}}$ **then**

9: $S_{\text{next}} \leftarrow \max(S_{\text{curr}} - 1, S_{\text{min}})$ $\triangleright$ Gradual Decrease

10: $C_{\text{stable}} \leftarrow 0$

11: **else**

12: $S_{\text{next}} \leftarrow S_{\text{curr}}$

13: $C_{\text{stable}} \leftarrow C_{\text{stable}} + 1$

14: **end if**

15: **else**

16: $S_{\text{next}} \leftarrow S_{\text{curr}}$

17: **if** $R \leq 0$ **then**

18: $C_{\text{stable}} \leftarrow C_{\text{stable}} + 1$

19: **else**

20: $C_{\text{stable}} \leftarrow 0$

21: **end if**

22: **end if**

23: **return** S_{next}

IV. EXPERIMENTAL RESULTS

Experiments were conducted to verify the proposed delay management method in an OpenAirInterface (OAI) nFAP End-to-End environment. Each reported throughput value represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial). The evaluation is designed to answer the following research questions:

- **RQ1**: Does the EWMA-based filtering effectively stabilize the noisy PNF Timing Info feedback under transport

network jitter?

- **RQ2:** Can the adaptive slots-ahead controller maintain stable throughput while minimizing unnecessary MAC HARQ RTT compared with static baselines?
- **RQ3:** Can the proposed closed-loop delay management preserve end-to-end synchronization stability and throughput under severe network congestion and dynamic jitter?

The rest of this section is structured as follows: Section IV-A outlines the hardware, software, and the automated rApp evaluation framework. Section IV-C addresses RQ1 by validating the EWMA mechanism. Section IV-D addresses RQ2 by benchmarking peak throughput and MAC HARQ RTT. Finally, Section IV-E addresses RQ3 by evaluating stability under injected delay congestion and unpredictable jitter.

A. Experimental Scenario and rApp Framework

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

TABLE I
TESTBED NODE CONFIGURATION

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (2026.w13)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
WIRELESS SYSTEM PARAMETERS

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

To ensure automated measurement and analysis with consistent data quality, an automated evaluation rApp was developed to operate on the Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [7]. As shown in Fig. 2, this rApp automates the start-up of the VNF and PNF. Once the gNB is established, it automatically connects to the control PC via SSH to toggle the UE's airplane mode using Android Debug Bridge (ADB), and subsequently uses SSH to launch the iPerf server on the Core Network (CN) server. It then executes a series of pre-scheduled iPerf tests automatically. Finally, the rApp collects all log files via O1 SFTP and utilizes Python scripts to plot the following data analysis charts. Crucially, it manages the network latency simulation at $T_5 - T_4$ and $T_2 - T_1$ via Linux Traffic Control (TC) to systematically validate the split-architecture stability.

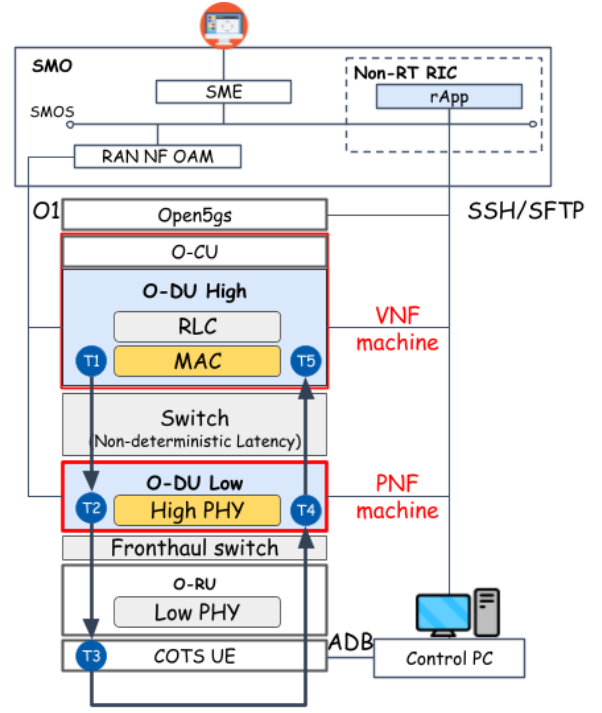


Fig. 2. Architecture of the rApp automated evaluation framework.

B. Thorough System-Level Validation and Generalization

To prove the stability, generalization capability, and compatibility of the proposed timing control and nFAPi modifications, our software implementation was submitted and successfully merged into the official upstream OpenAirInterface (OAI) repository. Beyond our localized end-to-end evaluation using the rApp framework, the proposed solution went through thorough testing in the official OAI Continuous Integration and Continuous Deployment (CI/CD) pipeline. Specifically, the code successfully passed all image building and cross-compilation tests across standard x86 and ARM64-based architectures, including enterprise-grade Red Hat Enterprise Linux (RHEL) clusters and NVIDIA Jetson edge platforms. Furthermore, our implementation successfully completed key system-level integration and physical layer regression tests. These tests include 5G Standalone (SA) end-to-end integration, real radio frequency (RF) transmissions using USRP B200 and AW2S radio units, standard F1/N2 interface control-plane handover procedures, and 3GPP-compliant physical layer channel simulations. These regression and compilation results confirm that our proposed software-defined timing loops are highly robust, backward-compatible, and ready for diverse commercial cloud hardware deployments without introducing any hardware dependency.

C. Scenario I: Validation of EWMA for PNF Timing Info

This scenario validates the application of an Exponentially Weighted Moving Average (EWMA) to smooth the PNF Timing Info (Δt_{arrive}). Fig. 3(a) shows the timing control directly using the raw PNF Timing Info (Δt_{arrive}), where the

raw timing values change a lot due to network jitter. Fig. 3(b) is the cumulative count of the packets exceeding the deadline when the timing adjustment directly uses the raw PNF Timing Info (Δt_{arrive}). The EWMA mechanism filters out the high-frequency jitter, as shown in the PNF Timing Info (Δt_{arrive}) after applying EWMA in Fig. 3(c). Finally, Fig. 3(d) shows the cumulative count of the packets exceeding the deadline under the EWMA-based timing control. The results show that applying EWMA allows the timing loop to adjust the slots ahead stably, which significantly reduces the number of packets that arrive after the deadline.

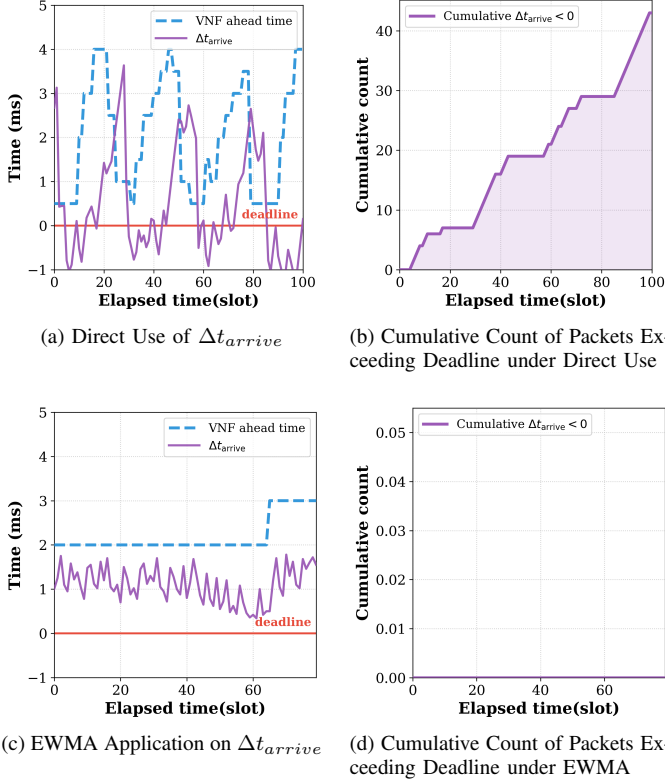


Fig. 3. Validation of EWMA processing on PNF Timing Info.

D. Scenario II: Performance Benchmark

We benchmark the peak throughput and MAC HARQ RTT against a static baseline. Under stable fronthaul conditions, as shown in Fig. 4, the proposed algorithm prioritizes guaranteeing the optimal throughput under different offered loads, consistently maintaining near-peak throughput performance. Simultaneously, while ensuring this optimal throughput, the algorithm also optimizes the selection of the slots ahead trigger offset to optimize the MAC layer HARQ Round-Trip Time (RTT), effectively avoiding unnecessary overhead.

E. Scenario III: Stability under Network Congestion

To evaluate stability, we inject delay and jitter between the VNF and PNF using Linux TC. As shown in Fig. 5a, static slots ahead configurations suffer from synchronization failures and throughput drops when delay exceeds the timing

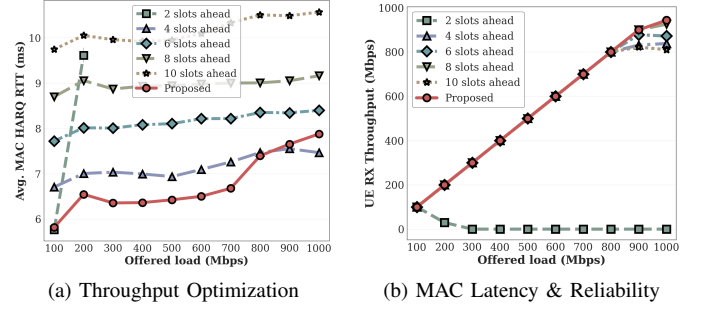


Fig. 4. Performance evaluation: MAC RTT efficiency and system throughput.

window, whereas our proposed adaptive mechanism dynamically increases the slots ahead to maintain stable throughput. Additionally, we evaluate the UE DL RX throughput under a saturated 1 Gbps UDP offered load with unpredictable link jitter. As shown in Fig. 5b, while fixed slot ahead configurations experience severe throughput degradation due to late packets, the proposed adaptive method dynamically tracks jitter and secures stable, high throughput at the UE.

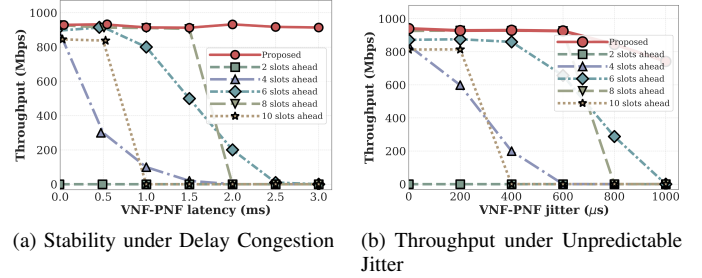


Fig. 5. System stability and throughput performance under varying network conditions.

V. CONCLUSION

This paper presented a dynamic timing adjustment framework for disaggregated RAN architectures, specifically focusing on the nFAPI-based Split Option 6 over non-ideal packet-switched networks. Recognizing the limitations of static slots ahead configurations under unpredictable network jitter and server processing delays, we proposed an Adaptive Slots Ahead mechanism based on Exponentially Weighted Moving Average (EWMA) to dynamically track and reduce network latency variations. Experimental validations conducted on the OpenAirInterface (OAI) platform with a commercial Pegatron O-RU demonstrated that the proposed method significantly reduces synchronization failures and maintains stable system throughput, even under severe network congestion. Unlike existing solutions that rely on expensive hardware-level synchronization or specialized acceleration, our software-based approach offers a cost-effective and flexible alternative for deploying robust O-RAN systems on general-purpose Commercial Off-the-Shelf (COTS) servers.

Although the proposed software-based timing control achieves robust performance, the current evaluation focuses

primarily on downlink timing control under controlled delay and jitter injection. Future work will extend the controller to joint uplink/downlink timing optimization, multi-cell deployments, and analytical stability bounds for the feedback loop.

REFERENCES

- [1] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, "X5g: An open, programmable, multi-vendor, end-to-end, private 5g o-ran testbed with nvidia arc and openair-interface," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11 305–11 322, 2025.
- [2] S.-Q. Lee and M.-S. Lee, "A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb," in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2022.
- [3] Y. Jia, Y. Zhong, M. Wang, J. Gao, P. Zhang, X. Liu, and X. Jin, "Aquifer: Transparent microsecond-scale scheduling for vRAN workloads," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3171–3184, 2024.
- [4] Small Cell Forum (SCF), "5g network fapi (nfapi) specification," Small Cell Forum (SCF), Tech. Rep. SCF225, 2020. [Online]. Available: <https://www.smallcellforum.org/docs/5g-nfapi-specifications/>
- [5] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.
- [6] X. Wang *et al.*, "An open-source implementation of the 5g nr functional split option 6," in *2022 IEEE International Conference on Communications (ICC)*, 2022, pp. 1–6.
- [7] O-RAN Alliance, "O-ran architecture description," O-RAN Alliance Working Group 1, Tech. Rep. O-RAN.WG1.O-RAN-Architecture-Description, 2024.